

MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 4

INTERMEDIATE

MVC PATTERN

ROMAN URDU

CHAPTER 4

REST API Architecture — MVC Pattern

Code ko professional tarike se organize karo — jaise real companies karte hain!

Is Chapter Mein Aap Seekhenge:

- MVC Pattern — Model, View, Controller architecture ka concept
- Professional folder structure banana
- Controllers — route logic ko alag file mein likhna
- Express Router — organized aur clean routing
- Async Error Handler — DRY code likhna
- Environment Config — development vs production
- ShopEase ka complete backend structure banana
- API Documentation — endpoints ka record rakhna
- Thorough API testing aur debugging

■ Chapters 1-3 Recap

Express server, routes, MongoDB, Mongoose models, CRUD — yeh sab aana chahiye. Chapter 4 mein hum yeh sab ko ek professional structure mein daalenge!

TABLE OF CONTENTS

1.0 MVC Pattern Kya Hai?

- 1.1 Model, View, Controller — har ek ka kaam
- 1.2 Bina MVC vs MVC — farq kya hai?
- 1.3 MERN mein MVC kaise apply hota hai

2.0 Professional Folder Structure

- 2.1 Poora project structure
- 2.2 Har folder ka maqsad
- 2.3 File naming conventions

3.0 Controllers Banana

- 3.1 Controller kya hota hai?
- 3.2 Product Controller
- 3.3 User Controller
- 3.4 Order Controller

4.0 Async Error Handler

- 4.1 DRY principle kya hai?
- 4.2 asyncHandler utility banana
- 4.3 Custom Error Class
- 4.4 Global error middleware update

5.0 Express Router — Clean Routing

- 5.1 Router kya hai?
- 5.2 productRoutes.js — final version
- 5.3 userRoutes.js
- 5.4 orderRoutes.js
- 5.5 server.js mein sab register karna

6.0 Environment Configuration

- 6.1 .env ka poora setup
- 6.2 config/config.js banana
- 6.3 Development vs Production settings

7.0 Final ShopEase Backend Structure

- 7.1 Poora folder tree
- 7.2 Saari files aur unka kaam

- 7.3 Import/Export chain

8.0 API Documentation

- 8.1 Documentation kyun zaroori?
- 8.2 REST API endpoints table
- 8.3 Request/Response format

9.0 Testing & Debugging

- 9.1 Thunder Client collections
- 9.2 Common errors aur solutions
- 9.3 Debugging tips

10.0 Summary & Practice

- 10.1 Chapter recap
- 10.2 Practice exercises
- 10.3 Chapter 5 preview

SECTION 1 — MVC Pattern Kya Hai?

1.0 — MVC Pattern Kya Hai?

Abhi tak humara code messy tha — routes, logic, database calls sab ek hi jagah. Jaise ek kamre mein bistar, kitchen, aur office sab hoon — kaam hota hai lekin bahut mushkil se! **MVC** ek design pattern hai jo code ko 3 hisso mein todta hai:

M	Model	Database se baat karta hai. Mongoose models — Product, User, Order. Data store, fetch, validate sab yahan.
V	View	User ko jo dikhai deta hai. MERN mein React yeh kaam karta hai — HTML pages. Backend se directly koi kaam nahi.
C	Controller	Dono ke beech mein director. Request aata hai -> Controller decide karta hai -> Model se data maango -> Response bhejo.

1.1 — Bina MVC vs MVC ke Saath

Masla	Bina MVC ■	MVC ke Saath ■
Code location	Sab server.js mein — 500+ lines	Alag files — 50-100 lines each
Bug dhundhna	Poora file chhanno	Sirf controller file dekho
Team work	Conflicts hote hain	Har banda alag file pe kaam
Testing	Mushkil — sab entangled	Controller alag test ho sakta
Code reuse	Duplicate code likhna padta	Controller function reuse karo
New feature add	Sab kuch toota ja sakta	Alag file mein safely add karo

1.2 — MERN mein MVC Ka Flow

User ne 'All Products' maange — parda ke peeche kya hoga:

■ USER	Browser pe /products click kiya
---------------	---------------------------------

■■ REACT	GET /api/products request bheji
■■ ROUTER	Request aayi — productRoutes pe bhejo
■ CONTROLLER	getProducts() function chala — logic yahan hai
■ MODEL	Product.find() — database se data maango
■■ MONGODB	Products ki list wapas di
■ CONTROLLER	Response format karo — { success, data }
■■ REACT	JSON mila — screen pe products dikhai diye! ■

■ TIP

MVC har jagah use hota hai — Netflix, Amazon, Facebook sab yeh pattern follow karte hain. Ek baar seekh lo — life bhar kaam aayega!

SECTION 2 — Professional Folder Structure

2.0 — Poora Project Structure

Ek professional MERN backend ka structure aisa hota hai. Har folder ka ek specific kaam hai — kuch bhi random nahi:

shopease/

■■■ config/

■ ■■■ db.js

■ ■■■ config.js

■■■ controllers/

■ ■■■ productController.js

■ ■■■ userController.js

■ ■■■ orderController.js

■■■ middleware/

■ ■■■ asyncHandler.js

■ ■■■ errorMiddleware.js

■ ■■■ authMiddleware.js

■■■ models/

■ ■■■ Product.js

■ ■■■ User.js

■ ■■■ Order.js

■■■ routes/

■ ■■■ productRoutes.js

■ ■■■ userRoutes.js

■ ■■■ orderRoutes.js

■■■ utils/

■ ■■■ helpers.js

■■■ .env

■■■ .gitignore

■■■ package.json

2.1 — Har Folder Ka Kaam

Folder	Color	Kya Hota Hai?
<code>config/</code>	Teal	Database connection, environment variables — app ki settings
<code>controllers/</code>	Yellow	Route logic — request process karo, DB se baat karo, response bhejo
<code>middleware/</code>	Orange	Request-response ke beech functions — auth, error handling, logging
<code>models/</code>	Purple	Mongoose schemas aur models — database ki shape define karo
<code>routes/</code>	Blue	URL endpoints define karo — controller functions se connect karo
<code>utils/</code>	Pink	Reusable helper functions — jaise token generate karna, email bhejna
<code>.env</code>	Red	Secret environment variables — KABHI GitHub pe mat daalo!
<code>server.js</code>	Gold	App ka main entry point — sab kuch yahan start hota hai

■ TIP

Folders banane ke liye terminal mein: `mkdir config controllers middleware models routes utils` — ya VS Code mein folder icon se! Structure pehle banao, phir files daalo.

SECTION 3 — Controllers Banana

3.0 — Controller Kya Hota Hai?

Controller ek file hai jisme saari **business logic** hoti hai. Routes sirf URL aur method define karte hain — actual kaam (DB query, validation, response) controller karta hai. Ek function = ek kaam. Clean, simple, testable!

■ TIP

Controller functions mein sirf yeh 3 cheezein hoti hain: (1) Request se data lo → (2) Model se kaam karo → (3) Response bhejo. Itna hi! Koi aur kaam nahi.

3.1 — Product Controller (controllers/productController.js)

■ controllers/productController.js

```
const asyncHandler = require('../middleware/asyncHandler'); // Section 4
                        mein banayenge

const Product      = require('../models/Product');

// ■■■ @desc    Get all products
// ■■■ @route    GET /api/products
// ■■■ @access   Public (koi bhi dekh sakta hai)

const getProducts = asyncHandler(async (req, res) => {
  // Query parameters

  const keyword = req.query.keyword
    ? { name: { $regex: req.query.keyword, $options: 'i' } }
    : {};

  const category = req.query.category ? { category: req.query.category }
  : {};

  const page     = Number(req.query.page) || 1;
  const limit    = Number(req.query.limit) || 12;
  const skip     = (page - 1) * limit;

  const filter   = { ...keyword, ...category };
  const total    = await Product.countDocuments(filter);
  const products = await Product.find(filter)
```

```

        .sort({ createdAt: -1 })
        .skip(skip)
        .limit(limit);

res.status(200).json({
  success: true,
  count: products.length,
  total,
  totalPages: Math.ceil(total / limit),
  currentPage: page,
  data: products,
});
});

// ■■■ @desc    Get single product by ID
// ■■■ @route    GET /api/products/:id
// ■■■ @access   Public

const getProductById = asyncHandler(async (req, res) => {
  const product = await Product.findById(req.params.id);
  if (!product) {
    res.status(404);
    throw new Error('Product nahi mila!');
  }
  res.status(200).json({ success: true, data: product });
});

// ■■■ @desc    Create new product
// ■■■ @route    POST /api/products
// ■■■ @access   Private/Admin

const createProduct = asyncHandler(async (req, res) => {
  const product = await Product.create(req.body);
  res.status(201).json({ success: true, data: product });
});

// ■■■ @desc    Update product
// ■■■ @route    PUT /api/products/:id
// ■■■ @access   Private/Admin

```

```

const updateProduct = asyncHandler(async (req, res) => {
  const product = await Product.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true, runValidators: true }
  );
  if (!product) { res.status(404); throw new Error('Product nahi mila!'); }
  res.status(200).json({ success: true, data: product });
});

// ■■■■ @desc    Delete product
// ■■■■ @route    DELETE /api/products/:id
// ■■■■ @access   Private/Admin

const deleteProduct = asyncHandler(async (req, res) => {
  const product = await Product.findByIdAndDelete(req.params.id);
  if (!product) { res.status(404); throw new Error('Product nahi mila!'); }
  res.status(200).json({ success: true, message: 'Product delete ho gaya!' });
});

// ■■■■ @desc    Create product review
// ■■■■ @route    POST /api/products/:id/reviews
// ■■■■ @access   Private

const createReview = asyncHandler(async (req, res) => {
  const { rating, comment } = req.body;
  const product = await Product.findById(req.params.id);
  if (!product) { res.status(404); throw new Error('Product nahi mila!'); }

  // Check: kya user ne pehle review diya?
  const alreadyReviewed = product.reviews.find(
    r => r.user.toString() === req.user._id.toString()
  );
  if (alreadyReviewed) {

```

```

        res.status(400); throw new Error('Aap pehle hi review de chuke hain!'
    );
    }

    const review = { user: req.user._id, name: req.user.name, rating: Number(
rating), comment };

    product.reviews.push(review);

    product.numReviews = product.reviews.length;

    product.rating = product.reviews.reduce((sum,r) => sum + r.rating, 0) /
product.reviews.length;

    await product.save();

    res.status(201).json({ success: true, message: 'Review add ho gaya!' })
;
    });

module.exports = { getProducts, getProductById, createProduct, updateProd
uct, deleteProduct, createReview };

```

3.2 — User Controller (controllers/userController.js)

■ controllers/userController.js

```

const asyncHandler = require('../middleware/asyncHandler');

const User          = require('../models/User');

const jwt           = require('jsonwebtoken'); // Chapter 5 mein install
kareenge

// JWT Token generate karne ka helper

const generateToken = (id) => {

    return jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: '30d' });
};

// ■■■ @desc    Register user
// ■■■ @route    POST /api/users/register
// ■■■ @access   Public

const registerUser = asyncHandler(async (req, res) => {

    const { name, email, password } = req.body;

    // Check: Email pehle se registered?

    const userExists = await User.findOne({ email });

```

```

    if (userExists) { res.status(400); throw new Error('Yeh email pehle se
registered hai!'); }

    const user = await User.create({ name, email, password });

    res.status(201).json({
      success: true,
      data: {
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        token: generateToken(user._id), // JWT token
      }
    });
  });

// ■■■ @desc    Login user
// ■■■ @route    POST /api/users/login
// ■■■ @access   Public

const loginUser = asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  // Password bhi select karo (schema mein select:false tha)
  const user = await User.findOne({ email }).select('+password');

  if (!user) { res.status(401); throw new Error('Email ya password galat
hai!'); }

  // bcrypt se password compare karo

  const isMatch = await user.comparePassword(password); // Chapter 5 mei
n add karenge

  if (!isMatch) { res.status(401); throw new Error('Email ya password gal
at hai!'); }

  res.status(200).json({
    success: true,
    data: { _id: user._id, name: user.name, email: user.email, role: user
.role,
      token: generateToken(user._id) }
  });
});

```

```

    });

});

// ■■■ @desc    Get user profile
// ■■■ @route    GET /api/users/profile
// ■■■ @access    Private (login zaroori)

const getUserProfile = asyncHandler(async (req, res) => {

    const user = await User.findById(req.user._id);

    if (!user) { res.status(404); throw new Error('User nahi mila!'); }

    res.status(200).json({ success: true, data: user });

});

// ■■■ @desc    Update user profile
// ■■■ @route    PUT /api/users/profile
// ■■■ @access    Private

const updateUserProfile = asyncHandler(async (req, res) => {

    const user = await User.findById(req.user._id);

    if (!user) { res.status(404); throw new Error('User nahi mila!'); }

    user.name    = req.body.name    || user.name;
    user.email    = req.body.email    || user.email;

    if (req.body.password) user.password = req.body.password;

    const updated = await user.save();

    res.status(200).json({

        success: true,

        data: { _id: updated._id, name: updated.name, email: updated.email,

            token: generateToken(updated._id) }

    });

});

module.exports = { registerUser, loginUser, getUserProfile, updateUserPro
file };

```

3.3 — Order Controller (controllers/orderController.js)

■ controllers/orderController.js

```

const asyncHandler = require('../middleware/asyncHandler');

const Order
    = require('../models/Order');

```

```

// ■■■ @desc    Create new order
// ■■■ @route    POST /api/orders
// ■■■ @access   Private

const createOrder = asyncHandler(async (req, res) => {
  const { orderItems, shippingAddress, paymentMethod,
    itemsPrice, taxPrice, shippingPrice, totalPrice } = req.body;

  if (!orderItems || orderItems.length === 0) {
    res.status(400); throw new Error('Koi order item nahi hai!');
  }

  const order = await Order.create({
    user: req.user._id, // logged in user ki ID
    orderItems, shippingAddress, paymentMethod,
    itemsPrice, taxPrice, shippingPrice, totalPrice,
  });

  res.status(201).json({ success: true, data: order });
});

// ■■■ @desc    Get logged in user's orders
// ■■■ @route    GET /api/orders/myorders
// ■■■ @access   Private

const getMyOrders = asyncHandler(async (req, res) => {
  const orders = await Order.find({ user: req.user._id }).sort({ createdAt: -1 });

  res.status(200).json({ success: true, count: orders.length, data: orders });
});

// ■■■ @desc    Get order by ID
// ■■■ @route    GET /api/orders/:id
// ■■■ @access   Private

const getOrderById = asyncHandler(async (req, res) => {
  const order = await Order.findById(req.params.id).populate('user', 'name email');

  if (!order) { res.status(404); throw new Error('Order nahi mila!'); }

  res.status(200).json({ success: true, data: order });
});

```

```

});

// ■■■■ @desc    Update order status (Admin)
// ■■■■ @route    PUT /api/orders/:id/status
// ■■■■ @access    Private/Admin

const updateOrderStatus = asyncHandler(async (req, res) => {
  const order = await Order.findById(req.params.id);
  if (!order) { res.status(404); throw new Error('Order nahi mila!'); }
  order.orderStatus = req.body.orderStatus || order.orderStatus;
  if (req.body.orderStatus === 'Delivered') {
    order.isDelivered = true;
    order.deliveredAt = Date.now();
  }
  const updated = await order.save();
  res.status(200).json({ success: true, data: updated });
});

// ■■■■ @desc    Get all orders (Admin)
// ■■■■ @route    GET /api/orders
// ■■■■ @access    Private/Admin

const getAllOrders = asyncHandler(async (req, res) => {
  const orders = await Order.find().populate('user', 'name email').sort({
    createdAt:-1 });
  res.status(200).json({ success: true, count: orders.length, data: orders });
});

module.exports = { createOrder, getMyOrders, getOrderById, updateOrderStatus, getAllOrders };

```

SECTION 4 — Async Error Handler (DRY Code)

4.0 — DRY Principle Kya Hai?

DRY = Don't Repeat Yourself. Chapter 3 mein har route mein try-catch likhte the — yeh repetitive tha. Ek utility function banate hain jo yeh kaam automatically kare — DRY principle!

4.1 — asyncHandler Utility (middleware/asyncHandler.js)

■ middleware/asyncHandler.js

```
// middleware/asyncHandler.js

// Yeh ek Higher Order Function hai:
// Ek function jo doosra function return karta hai
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

// Matlab kya hai?
// asyncHandler(fn) likhne par:
// 1. fn ko Promise mein wrap karo
// 2. Agar koi error aaye toh next(error) call karo
// 3. next(error) global error handler tak pohoncha deta hai

module.exports = asyncHandler;
```

Fayda samjho — pehle vs baad:

```
// ■ PEHLE – har function mein try-catch
const getProducts = async (req, res) => {
  try {
    const products = await Product.find();
    res.json({ data: products });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

```
// ■ BAAD – asyncHandler ke saath – clean!

const getProducts = asyncHandler(async (req, res) => {

  const products = await Product.find();

  res.json({ data: products });

  // Error aayega toh asyncHandler pakad lega!

});
```

4.2 — Custom Error Class (middleware/errorMiddleware.js)

Ek custom Error class banao — status code bhi error ke saath bhejne ke liye:

■ middleware/errorMiddleware.js

```
// middleware/errorMiddleware.js

// Custom Error Class – status code ke saath

class AppError extends Error {

  constructor(message, statusCode) {

    super(message); // Parent Error class ka constructor

    this.statusCode = statusCode;

    this.isOperational = true; // Humara error, not a bug

    Error.captureStackTrace(this, this.constructor);

  }

}

// Global Error Handler Middleware

// 4 parameters hone chahiye – Express ise error middleware samjhata hai

const errorHandler = (err, req, res, next) => {

  let statusCode = err.statusCode || res.statusCode || 500;

  let message = err.message || 'Kuch galat ho gaya!';

  // Mongoose: Invalid ObjectId (galat ID format)

  if (err.name === 'CastError' && err.kind === 'ObjectId') {

    statusCode = 404;

    message = 'Resource nahi mila – ID format galat hai!';

  }

  // Mongoose: Duplicate key (unique field repeat)

  if (err.code === 11000) {
```

```

    statusCode = 400;

    const field = Object.keys(err.keyValue)[0];
    message = `${field} pehle se exist karta hai!`;
  }

  // Mongoose: Validation error
  if (err.name === 'ValidationError') {
    statusCode = 400;
    message = Object.values(err.errors).map(e => e.message).join(', ');
  }

  // JWT errors (Chapter 5 mein kaam aayega)
  if (err.name === 'JsonWebTokenError') {
    statusCode = 401; message = 'Invalid token – dobara login karo!';
  }
  if (err.name === 'TokenExpiredError') {
    statusCode = 401; message = 'Token expire ho gaya – dobara login karo
!';
  }

  res.status(statusCode).json({
    success: false,
    message,
    // Development mein stack trace dikhai dega
    stack: process.env.NODE_ENV === 'development' ? err.stack : undefined
  });
};

// 404 Not Found handler
const notFound = (req, res, next) => {
  const error = new AppError(`Route nahi mila: ${req.originalUrl}`, 404);
  next(error);
};

module.exports = { errorHandler, notFound, AppError };

```

■ TIP

AppError class isliye useful hai ke hum controller mein likha sakte hain: throw new AppError('Product nahi mila!', 404) — aur error handler automatically sahi status code ke saath response bhejega!

SECTION 5 — Express Router — Clean Routing

5.0 — Route Files Ka Kaam

Route files sirf yeh 2 kaam karti hain: (1) URL + HTTP method define karo, (2) Controller function se connect karo. Aur kuch nahi! Logic bilkul nahi — woh controller mein hai.

5.1 — Product Routes (routes/productRoutes.js)

■ routes/productRoutes.js

```
const express = require('express');
const router = express.Router();
const {
  getProducts, getProductById, createProduct,
  updateProduct, deleteProduct, createReview
} = require('../controllers/productController');

// Middleware — Chapter 5 mein banayenge (abhi placeholder)
// const { protect, admin } = require('../middleware/authMiddleware');

// Public routes — koi bhi access kar sakta hai
router.get('/', getProducts);
router.get('/:id', getProductById);

// Protected routes — sirf logged in user
// router.post('/:id/reviews', protect, createReview);
router.post('/:id/reviews', createReview); // Abhi ke liye bina auth

// Admin routes — sirf admin
// router.post('/', protect, admin, createProduct);
// router.put('/:id', protect, admin, updateProduct);
// router.delete('/:id', protect, admin, deleteProduct);
router.post('/', createProduct); // Abhi ke liye bina auth
router.put('/:id', updateProduct);
router.delete('/:id', deleteProduct);
```

```
module.exports = router;
```

5.2 — User Routes (routes/userRoutes.js)

■ routes/userRoutes.js

```
const express = require('express');
const router = express.Router();

const {
  registerUser, loginUser, getUserProfile, updateUserProfile
} = require('../controllers/userController');

// Public routes

router.post('/register', registerUser);
router.post('/login', loginUser);

// Protected routes (Chapter 5 mein protect middleware add karenge)
router.get('/profile', getUserProfile);
router.put('/profile', updateUserProfile);

module.exports = router;
```

5.3 — Order Routes (routes/orderRoutes.js)

■ routes/orderRoutes.js

```
const express = require('express');
const router = express.Router();

const {
  createOrder, getMyOrders, getOrderById,
  updateOrderStatus, getAllOrders
} = require('../controllers/orderController');

// User routes

router.post('/', createOrder);
router.get('/myorders', getMyOrders);
router.get('/:id', getOrderById);

// Admin routes

router.get('/', getAllOrders);
```

```
router.put('/:id/status', updateOrderStatus);

module.exports = router;
```

5.4 — server.js — Final Clean Version

■ server.js

[illegible]

[illegible]

■ SAHI KIYA

server.js bilkul clean hai — sirf 40 lines! Sab organized hai. Routes, controllers, middleware sab alag files mein hain. Yahi professional backend ka structure hai.

SECTION 6 — Environment Configuration

6.0 — Environment Variables Kya Hain?

Real apps do jagah pe chalta hai: tumhara computer (**development**) aur live internet pe (**production**). Dono ki settings alag hoti hain — database alag, URLs alag, debug mode alag. Environment variables yeh differences handle karte hain.

6.1 — Complete .env File

■ .env (KABHI GITHUB PE MAT DAALO!)

[illegible]

6.2 — config/config.js — Centralized Config

Ek centralized config file banao — sab settings ek jagah:

■ config/config.js

```
// config/config.js

// Saari environment variables yahan se access karo

module.exports = {

  // Server

  port:      process.env.PORT      || 5000,
  nodeEnv:    process.env.NODE_ENV  || 'development',
  isDev:      process.env.NODE_ENV  !== 'production',

  // Database

  mongoUri:   process.env.MONGO_URI,

  // JWT

  jwtSecret:  process.env.JWT_SECRET || 'fallback_secret',
  jwtExpire:  process.env.JWT_EXPIRE || '30d',

  // Client

  clientUrl:  process.env.CLIENT_URL || 'http://localhost:3000',
};

// Usage: doosri files mein

// const config = require('./config/config');

// console.log(config.port); // 5000
```

6.3 — Development vs Production

Setting	Development	Production
NODE_ENV	development	production
Error Stack	Dikhata hai (debugging)	Nahi dikhata (security)
DB	Local ya Atlas free	Atlas paid ya dedicated
Logging	Verbose — sab print	Errors only
CORS	localhost:3000 allow	Domain specific
JWT Secret	Simple string	Long random string — very secure
npm run	npm run dev (nodemon)	npm start (node)

■ YAAD RAHO

Production mein deploy karte waqt .env variables hosting platform pe set karte hain (Render, Railway, Vercel) — .env file upload nahi karte. Hosting platforms ka apna environment variables section hota hai.

SECTION 7 — ShopEase Final Backend Structure

7.0 — Import/Export Chain — Sab Kaise Juda Hai

Ek request ke safar ko trace karte hain — GET /api/products pe kya hota hai:

1	<code>server.js</code>	Request aati hai /api/products -> productRoutes ko bhejo
2	<code>routes/productRoutes.js</code>	GET '/' -> getProducts controller function call karo
3	<code>controllers/productController.js</code>	getProducts() -> asyncHandler wrap mein
4	<code>middleware/asyncHandler.js</code>	try-catch automatically — errors pakdo
5	<code>models/Product.js</code>	Product.find() -> Mongoose MongoDB se query karta hai
6	MongoDB Atlas	Data wapas aata hai -> Mongoose object mein convert
5	<code>models/Product.js</code>	Mongoose data return karta hai
3	<code>controllers/productController.js</code>	res.json({ success:true, data:products }) bhejo
X	Browser/Thunder Client	JSON response milta hai — kaam ho gaya! ■

7.1 — Poori Files Summary

File	Lines	Kaam
<code>server.js</code>	~40	Entry point — middleware, routes, server start
<code>config/db.js</code>	~20	MongoDB connection function
<code>config/config.js</code>	~20	Centralized environment variables
<code>models/Product.js</code>	~60	Product schema — fields, validation, timestamps
<code>models/User.js</code>	~50	User schema — password hashing, select:false

models/Order.js	~55	Order schema — items, payment, delivery
controllers/productController.js	~60	5 functions: getAll, getOne, create, update, delete
controllers/userController.js	~80	4 functions: register, login, profile get/update
controllers/orderController.js	~80	5 functions: create, myOrders, getOne, updateStatus, getAll
middleware/asyncHandler.js	~5	try-catch wrapper — DRY error handling
middleware/errorMiddleware.js	~50	Global error handler + 404 handler + AppError class
routes/productRoutes.js	~20	Product endpoints — controller se connect
routes/userRoutes.js	~15	User endpoints — controller se connect
routes/orderRoutes.js	~15	Order endpoints — controller se connect

SECTION 8 — API Documentation

8.0 — Documentation Kyun Likhein?

Documentation ek reference guide hai — tumhare liye aur team ke liye. 'Kaun sa endpoint kya karta hai, kya bhejhna hai, kya aayega' — yeh sab ek jagah hona chahiye. Real companies REST API docs zaroor maintain karti hain.

8.1 — Product API Endpoints

Method	Endpoint	Auth	Description	Request Body
GET	/api/products	Public	All products (filter/sort/pagination) — query params: ?keyword=&category=&page=	None
GET	/api/products/:id	Public	Single product by ID	None
POST	/api/products	Admin	Create new product	{ name, description, price, category, images, stock }
PUT	/api/products/:id	Admin	Update product	Any product fields to update
DELETE	/api/products/:id	Admin	Delete product	None
POST	/api/products/:id/reviews	User	Add review to product	{ rating, comment }

8.2 — User API Endpoints

Method	Endpoint	Auth	Description
POST	/api/users/register	Public	Register new user — returns JWT token
POST	/api/users/login	Public	Login — returns JWT token
GET	/api/users/profile	User	Get logged in user's profile
PUT	/api/users/profile	User	Update profile — name, email, password

8.3 — Order API Endpoints

Method	Endpoint	Auth	Description
POST	/api/orders	User	Place new order
GET	/api/orders/myorders	User	Get logged in user's orders
GET	/api/orders/:id	User	Get single order details

GET	/api/orders	Admin	Get all orders
PUT	/api/orders/:id/status	Admin	Update order status

SECTION 9 — Testing & Debugging

9.0 — Thunder Client Collections

Thunder Client mein Collections banao — organized API testing ke liye. Ek Collection = ek group of related requests:

9.1 — Test Sequence (Is Order Mein Karo)

#	Test	Method	URL	Body?
1	Server health check	GET	/api/health	None
2	Create test product	POST	/api/products	JSON: name, price, category...
3	Get all products	GET	/api/products	None
4	Get single product	GET	/api/products/:id	None (ID URL mein)
5	Update product price	PUT	/api/products/:id	JSON: { price: 9999 }
6	Register user	POST	/api/users/register	JSON: name, email, password
7	Login user	POST	/api/users/login	JSON: email, password
8	Get profile	GET	/api/users/profile	None (Auth header mein token)
9	Create order	POST	/api/orders	JSON: orderItems, shipping...
10	Get my orders	GET	/api/orders/myorders	None
11	Delete product	DELETE	/api/products/:id	None
12	Get deleted product (404)	GET	/api/products/:id	404 error aana chahiye

9.2 — Common Errors aur Solutions

Error	Reason	Solution
Cannot GET /api/products	Route register nahi hua	server.js mein app.use('/api/products', productRoutes) check karo
MongoServerError: bad credentials	DB username/password galat	.env mein MONGO_URI check karo — username:password sahi hai?
req.body undefined	express.json() nahi lagaya	app.use(express.json()) routes se pehle add karo
CastError: Cast to ObjectId failed	Invalid MongoDB ID format	ID format check karo — 24 hex chars honi chahiye
ValidationError: name is required	Required field missing	Request body mein sab required fields check karo

CORS error in browser	CORS middleware nahi	app.use(cors()) express.json() se pehle add karo
Cannot find module '...'	Wrong import path	Relative path check karo — ../models/Product ya ./models/Product
Port 5000 already in use	Purana process chal raha	Terminal mein Ctrl+C karo ya: kill -9 \$(lsof -ti:5000)

SECTION 10 — Summary, Practice & Next Steps

10.0 — Chapter 4 Ka Recap

■ MVC Pattern	Model-View-Controller — code 3 hisson mein organize karo
■ Folder Structure	config, controllers, middleware, models, routes, utils
■ Controllers	Business logic — route se alag, clean functions
■ asyncHandler	DRY — try-catch har jagah likhne ki zaroorat nahi
■ AppError Class	Custom error — message + statusCode ek saath
■ errorHandler MW	Global error handling — CastError, ValidationError, duplicate key
■ productController	getProducts, getProductById, create, update, delete, review
■ userController	registerUser, loginUser, getProfile, updateProfile
■ orderController	createOrder, getMyOrders, getOrderById, updateStatus, getAll
■ Route Files	Sirf URL + method + controller — koi logic nahi!
■ server.js	Clean 40 lines — middleware, routes, error handlers
■ .env	PORT, MONGO_URI, JWT_SECRET, CLIENT_URL — sab secrets
■ config/config.js	Centralized settings — ek jagah se sab access
■ API Docs	Endpoints table — method, URL, auth, body — record rakhna
■ Error Solutions	8 common errors aur unke solutions — debugging toolkit

10.1 — Practice Exercises

■ Easy Level

1. Folder structure banao — config, controllers, middleware, models, routes, utils
2. asyncHandler.js banao aur product routes mein use karo
3. errorHandler.js banao — global handler aur 404
4. server.js ko clean version mein rewrite karo (40 lines)

■ Medium Level

1. productController.js banao — saare 5+1 functions
2. productRoutes.js update karo — controller functions connect karo
3. Thunder Client se saari product endpoints test karo
4. userController.js banao — register aur login (JWT baad mein)

■ Challenge Level

1. orderController.js banao — saare 5 functions
2. Saari 3 route files banao — clean, controller se connected
3. config/config.js banao — centralized settings
4. API documentation table banao — apne notes mein
5. Saari 12 test cases Thunder Client se run karo
6. Ek common error deliberately karo aur error handler ka response dekho

10.2 — Chapter 5 Ka Preview

Authentication & Authorization — JWT aur bcrypt

- bcrypt kya hai — password hashing ka concept
- JWT (JSON Web Token) — kya hai, kaise kaam karta hai
- User register — password hash karke save karna
- User login — password compare, JWT generate karna
- authMiddleware — protected routes banana
- Admin middleware — role-based access control
- Token refresh aur logout handle karna

Wah! Chapter 4 Mukammal! ■■■

Tumne aaj professional MVC architecture seekhi, 3 complete controllers likhe, async error handler banaya, aur poora backend ek clean structure mein organize kiya. Yeh wahi code structure hai jo Netflix, Uber, aur bari companies follow karti hain! **Chapter 5 mein security aayegi — JWT aur bcrypt ke saath!**